

SOFTWAREDESIGN EN -KWALITEIT

MODELLEREN

avans
hogeschool



SOFTWAREDESIGN EN -KWALITEIT

Doel:

Software zo ontwerpen dat onderhoudbaarheid gewaarborgd blijft.

Bestaande code te herschrijven zodat de onderhoudbaarheid verbetert.

ORGANISATIE

- 4 lessen voor LU1
- Theoretisch en Praktisch tentamen in LU1 (LW4+LW5).
- Herkansing (LW9+LW10)
- 6 lessen voor LU2
- Praktisch tentamen in LU2 (LW9)
- Herkansing (LW11)

Werken in 4-tallen

LEERUITKOMST 1

1: Modelleren

2: Modelleren binnen een architectuur

3: Coupling

4: Doorontwikkeling

Vervolg voor LU2

NA DEZE LES ...

1. Kun je een eenvoudig ontwerp middels een UML Klassendiagram maken.
2. Kun je de SOLID designprincipes benoemen

PLAATSBEPALING DESIGN

- ☐ Welke activiteiten voor softwareontwikkeling ken je?
- ☐ Wat is het nut van een goed ontwerp?
- ☐ Design & Architecture, voor nu:

- **Design** is:

- o software ontwerp op detail-niveau
- o 'dichtbij' code

- **Architectuur** is:

- o Logical viewpoint
- o Ontwerp op globaler niveau, bv systeem als geheel
- o Opdeling van geheel in onderdelen/subsystemen



QUIZ



OPDRACHT: ONTWERP EEN QUIZ

☐ Ontwerp:

- UML klassendiagram + Opzet voor GUI

☐ Requirements:

- Kennisquiz over landen en hun hoofdstad
- Meerkeuze vragen
- Vraag: De hoofdstad van Nederland is...
- Lijst van 4 mogelijke antwoorden
- Een button om het antwoord te controleren
- Feedback:
 - Ja goed geantwoord! / Nee, het antwoord was helaas fout...

KLASSENDIAGRAM:

OPDRACHT: VERANDERING

Wat moeten we aanpassen wanneer ik

- ☐ een andere GUI wil?
- ☐ een puntentelling #goed / #vragen wil?
- ☐ die wil uitbreiden met een bonus wanneer alle vragen goed?
- ☐ de tijd wil bijhouden en snelle antwoorden hoger beloon?
- ☐ feedback op de vragen wil (correcte antwoorden) aan het einde?
 - deze moet kunnen mailen?
 - of kunnen op mijn facebook zetten?

Waar komen de vragen eigenlijk vandaan?

- ☐ Database? / Tekst-file? / Excel-sheet? / Web?

KLASSENDIAGRAM:

DESIGN FOR CHANGE

- ☐ Met een goed ontwerp ben je beter in staat veranderingen in je software te kunnen doorvoeren
- ☐ Lukt je dat niet meer... dan sterft je software
 - Aanpassingen doorvoeren
 - wordt te moeilijk
 - wordt te duur
 - duurt te lang
 - ...
 - En dan?
- ☐ Met design for change houd je de software levend

**OUDER WORDEN
IS
EIGENLIJK
BEST VERNIEUWEND**

de oma van
Loesje
Postbus 1045
6801 BA Arnhem
www.loesje.nl

DESIGN SMELLS

☐ Rigidity

- Aanpassingen zijn moeilijk
- Sneeuwbaaleffect: een kleine aanpassing resulteert in een lawine aan veranderingen

☐ Fragility

- Bij een enkele aanpassing, breekt de software op vele plaatsen

☐ Complexity

- Methoden te lang, te veel logica in 1 methode (of klasse)

☐ Opacity

- Moeilijk te begrijpen

DESIGN PRINCIPES

Een goed ontwerp voldoet aan design principes (en dat is niet altijd even gemakkelijk)

Voor nu:

- ☐ Single responsibility principle (S in SOLID)

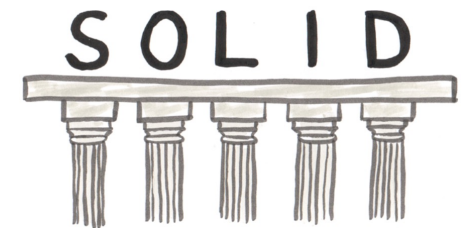
- A class should have one, and only one, reason to change

- ☐ Open closed principle (O in SOLID)

- A class should be open for extending its behavior, closed for modifications

- ☐ Program to an interface, not an implementation

- 'interface': interface óf abstract class



<https://www.freecodecamp.org/news/solid-principles-explained-in-plain-english>

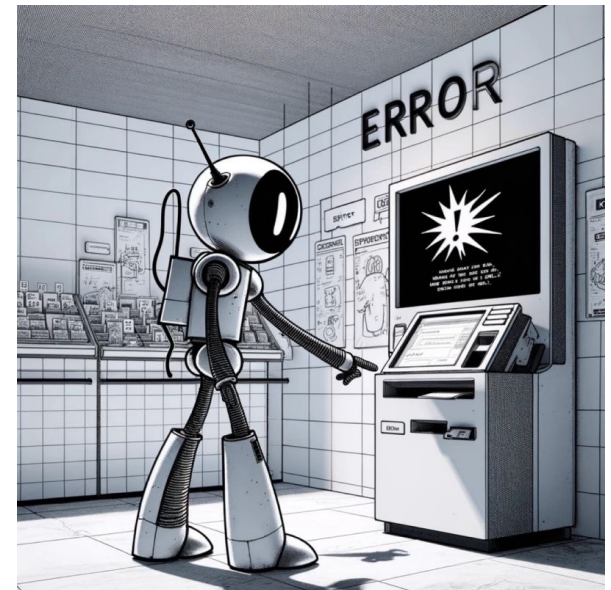
DESIGN GOALS

Er zijn verschillende doelen mogelijk bij het maken van een ontwerp

□ Bijvoorbeeld

- Modifiability
- Extensibility
- Reusability
- Testability
- Security
- Efficiency (e.g. transaction rate, network usage)

Let op: deze doelen kunnen ook tegenstrijdig zijn



HUISWERK

Zie opdrachtomschrijving op Brightspace

Deadline: begin volgende les

avans
hogeschool



***VOLGENDE
KEER***

**Volgende keer
gaan we werken
aan ontwerpen
volgens een
architectuur.**